

MARI: A Motion-Aware Adaptive Range Index for Exact Range Reporting over Bounded-Drift Integer Keys

Shah Nadim Kamran Rian^{1*} and Saheeb Tareque^{1†}

^{1*}Department of Electrical and Computer Engineering, North South University, Dhaka, Bangladesh.

*Corresponding author(s). E-mail(s): shah.rian.252@northsouth.edu;
Contributing authors: saheeb.tareque.252@northsouth.edu;

[†]These authors contributed equally to this work.

Abstract

Many applications maintain integer-valued keys that change frequently while requiring exact range queries, such as “which items currently lie between 4,800 and 4,900 points?”. An ordered index answers such queries efficiently, but each value change requires deleting the old entry and inserting the new one. Under update-intensive workloads, this relocation cost, rather than the query itself, dominates index maintenance. Our starting observation is that in many of these workloads the movement is small and bounded: between two consecutive updates a key rarely jumps far, obeying $|\Delta k| \leq \delta$ for a modest δ . We call this bounded drift and build an index around it. **MARI** gives every bucket a narrow core and a wider *guard*; a key that drifts but stays inside its guard is simply appended in place instead of relocated, and every query result is checked against a small authoritative table so the answer is always exact. On a reference implementation that we verify against a brute-force oracle on every run, MARI relocates **0.024** items per update where faithful ART, PGM, and Bx-tree baselines all relocate about **0.990**, roughly a **40×** reduction, at the lowest CPU time of any baseline and with a modest, clearly-stated memory overhead. Because updates are appended rather than relocated, MARI also writes far fewer durable bytes per update than native storage engines under per-update synchronous commits: about **1400×** fewer than a memory-mapped B+-tree (LMDB) and **4.2×** fewer than an LSM engine (RocksDB). We confirm that bounded drift is a real property across five unrelated domains. All measurements come from a single-machine Python prototype and are reported as implementation-independent counts; a native multicore evaluation remains future work.

Keywords: range reporting, updatable index, bounded drift, write amplification, exact indexing, learned indexes

1 Introduction

Answering a range query over integer keys, returning every item whose key falls in $[a, b]$, is one of the oldest and best-understood problems in data management. When the keys are fixed, a B+-tree or a radix tree settles it. The setting we care about is different: the key we are indexing is itself a moving target, and the query has to reflect the value *right now*, with no room for error.

Motivating application

Consider an exchange surveillance system that must continuously identify which listed securities are trading inside a given price band $[a, b]$. The indexed key is the price in cents, which changes on every tick. What makes the problem tractable is that the movement is bounded by market regulation. Under the U.S. Securities and Exchange Commission’s Limit Up–Limit Down mechanism, a Tier-1 security priced above three dollars cannot trade more than five percent from its rolling five-minute reference price before the exchange pauses it. The trading mechanism therefore imposes a bound $|\Delta k| \leq \delta$, where δ is fixed by regulation rather than assumed. The query still has to be exact. A surveillance alert that misses a symbol, or fires on one just outside the band, is worse than no alert at all. The update rate is high, and the key is a single integer. The same shape turns up far from finance: a leaderboard where a match nudges a score by a bounded step, or an Elo service where the K -factor caps how far one game can move a rating.

Limitations of existing approaches

Ordered indexes support exact range queries efficiently, but every value change requires a `delete(old)` followed by an `insert(new)`, a *relocation*. Under update-intensive workloads these relocations dominate maintenance cost through random writes, cache misses, and lock contention. Moving-object indexes such as the TPR-tree and Bx-tree absorb value changes, but they are designed for predictive, multi-dimensional queries, and their one-dimensional linearizations admit false hits, so exactness is lost by construction. Learned indexes and database cracking adapt impressively, but they adapt to the query workload or the key *distribution*, not to *motion*. To our knowledge, no existing structure satisfies all four of the following simultaneously: (i) exact range answers; (ii) one-dimensional integer keys arriving at a high update rate; (iii) an assumption of bounded drift; and (iv) a motion-aware update that absorbs a change without relocating the entry. That conjunction is the gap we set out to fill.

Contributions

This paper makes four contributions.

- We formalize exact range reporting over bounded-drift streaming integer keys, keeping clearly separate what the index *guarantees* and what the workload *provides* (Section 3).
- We propose MARI, whose central idea is *guarded bucket ownership*: a per-bucket guard that lets most updates land in place, an authoritative table T that keeps queries exact, and per-bucket versioned deltas held small by a compaction policy.
- We prove the cost bounds we rely on (amortized $O(1/\varepsilon)$ compaction, $O(1)$ maintenance, $O(n + m)$ space) and give a *relocation-scan coupling* result that pinpoints why single-width ordered indexes are stuck relocating on almost every change while MARI is not.
- We build a reference implementation, check it for exactness against a brute-force oracle on every run, compare it with faithful ART, PGM, and Bx-tree baselines, and validate the bounded-drift assumption on five real datasets from unrelated domains.

2 Related Work

MARI draws on four lines of prior work: ordered indexes, moving-object indexes, write-optimized structures, and adaptive or learned indexes.

2.1 Ordered and updatable indexes

The B+-tree and its descendants are the natural home for range queries, and much effort has gone into making them tolerate high update rates, from partitioned B-trees to write-friendly variants [1, 2]. The adaptive radix tree (ART) [3] and the latch-free Bw-tree [4] push main-memory and multicore performance further. All of them answer exact range queries; none escapes the fact that changing a key means moving its entry.

2.2 Moving-object and time-parameterized indexes

A separate community has long indexed values that change over time. The TPR-tree [5] and TPR*-tree [6] store motion functions and answer predictive queries; the Bx-tree [7], B^{dual}-tree [8], and ST²B-tree [9] map moving points onto a B+-tree through a space-filling curve. These structures absorb change gracefully, but they are built for prediction in two or more dimensions, and the one-dimensional linearizations return candidate sets that must be re-checked or enlarged, causing false hits that break exactness.

2.3 Write-optimized indexes

Differential files [10] and, most influentially, the log-structured merge-tree [11] trade read cost for write throughput by buffering updates and merging them in batches. MARI inherits the spirit of this idea, append rather than overwrite, but the LSM buffer is global, whereas our deltas are scoped to a bucket and tied to that bucket’s guard, which is what lets us reason about locality and relocation.

2.4 Adaptive and learned indexes

Database cracking [12–14] refines the physical layout in response to the queries it sees, and learned indexes [15–18] replace search with a model of the key distribution. Both are genuinely adaptive, but neither adapts to *motion*: they assume the data they have organized will not move.

A more distant relative is the theory of *kinetic data structures* [19, 20], which maintains combinatorial attributes of objects moving along known trajectories. We borrow the mindset, reason about movement rather than fight it, but our setting differs: we assume only a per-step displacement bound, not a trajectory, and we maintain an exact one-dimensional range index rather than a geometric certificate.

The upshot is summarized in Table 1. Read down the columns and a pattern appears: each family checks some boxes and misses others, and the particular combination our workload needs, exact, current-time, cheap to update, motion-aware, one-dimensional, is covered by no single prior approach.

Table 1 Comparison of MARI with related indexing approaches. ✓ = requirement met; ✗ = not met. Only MARI satisfies all requirements.

Approach	Exact range	Current-time (not pred.)	Cheap update	In-place under motion	1-D int. keys
B+-tree / ART / Bw-tree [3, 4]	✓	✓	✗	✗	✓
Moving-object (TPR / Bx-tree) [5, 7]	✗*	✗	✓	✓	✗
Cracking / Learned [12, 16, 17]	✓	✓	✗	✗	✓
LSM-tree [11]	✓	✓	✓	✗	✓
Kinetic data structures [19]	✓	✓	✓	✗	✗
MARI (this work)	✓	✓	✓	✓	✓

* One-dimensional space-filling-curve moving-object indexes return false hits and require query enlargement for guaranteed recall; exactness is therefore lost by construction.

3 Methodology

We first separate the correctness guarantee from the workload assumption, then describe the index structure, the update and query operations, and the complexity bounds.

3.1 Exactness and bounded drift

Definition 1 (Exact range reporting) For a query $[a, b]$, let $R = \{i : a \leq \text{key}(i) \leq b\}$ be the true set of items at the current instant. An index is *exact* if the set A it returns equals R , no false positives and no false negatives, for every query against every state the workload can reach.

Definition 2 (Bounded drift) A key stream has *bounded drift* δ if, between consecutive updates to the same item, its key moves by at most δ ; that is, $|\Delta k| = |k_{t+1} - k_t| \leq \delta$.

This is the assumption, and it is a property of the *workload*, not something the index enforces. We audit it rather than trust it blindly (Section 4), and MARI stays exact even where it is violated, a violation simply costs more relocations, never correctness.

3.2 Index structure

MARI cuts the key universe into fixed-width buckets and layers five structures on top; the whole mechanism is drawn in Figure 1.

1. **Guarded buckets.** Bucket b owns a narrow *core* $C_b = [bw, (b+1)w)$ and a wider *guard* $G_b = [bw-g, (b+1)w-1+g]$. The two widths, w and g , are set *independently*, a choice that turns out to be the crux of the whole approach (Proposition 3).
2. **The authoritative table T .** A map from item id to (key, bucket). It is the single source of truth, consulted to verify query candidates. We deliberately do not persist it: after a crash it is rebuilt in one pass over the logs, which keeps the write path cheap.
3. **The stable tier S_b .** A sorted per-bucket list that supports the ordered scan a range query needs.
4. **The delta log D_b .** An append-only per-bucket log recording in-place updates and tombstones (the latter mark a key that has migrated away).
5. **Overflow chains.** A mechanism for pathological hotspots: if a single value gathers more copies than a bucket-split can separate, those ids move to a dedicated chain keyed by that exact value.

3.3 Update and query operations

Both paths appear as a flowchart in Figure 2; the prose here explains the reasoning behind them.

Update. Given an item's new key k' , we read its current bucket b from T and ask one question: is k' still inside the guard G_b ? If it is, which, under bounded drift, is overwhelmingly the common case, we do nothing structural beyond appending (id, k') to that bucket's delta. No node splits, no re-balancing, no movement. Only when k' has escaped the guard do we migrate: write a tombstone in D_b and insert into the destination bucket $D_{b'}$ with $b' = \lfloor k'/w \rfloor$. Either way we refresh T , and we compact a bucket j once its delta has grown to an ε -fraction of its stable tier ($|D_j| \geq \varepsilon|S_j|$), folding the deltas back into sorted order and dropping tombstoned entries.

Query. To answer $[a, b]$ we first select only the buckets whose *guard* overlaps the interval, a superset of the buckets that could contribute, because a key may sit in a bucket whose core lies outside $[a, b]$ but whose guard does not. We scan those buckets' stable and delta tiers to gather candidates, and then, importantly, we *verify* each candidate i against T : it enters the answer if and only if $a \leq T[i].\text{key} \leq b$. That verification step is what lets us tolerate stale positions without ever returning a wrong answer.

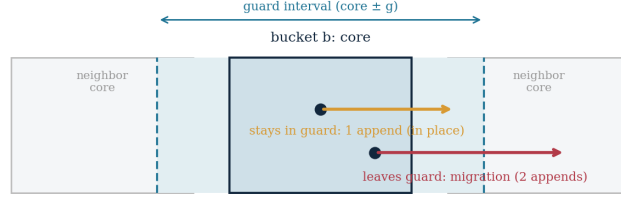


Fig. 1 Guarded bucket ownership. Each bucket owns a narrow core and a wider guard; a key that drifts within its guard (the dot moving inside the shaded band) is updated in place rather than relocated. The authoritative table T and the per-bucket delta together keep queries exact.

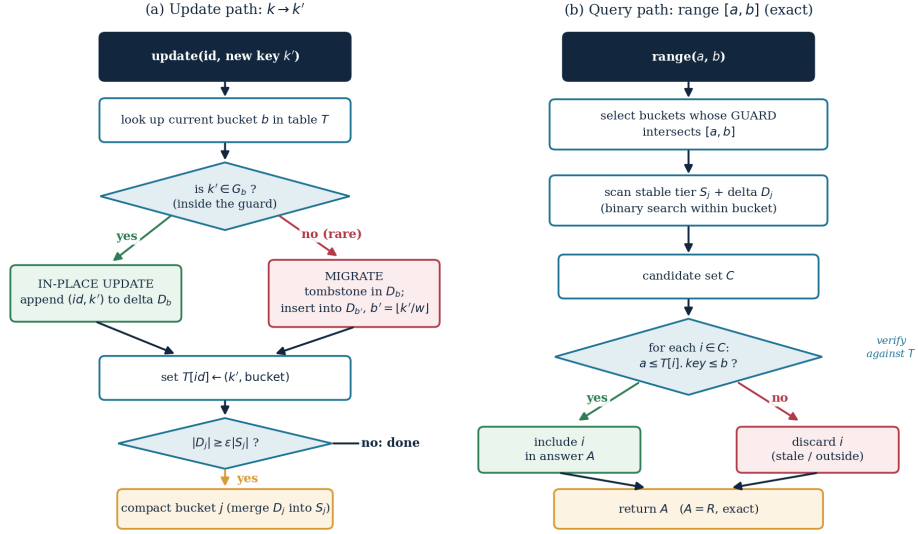


Fig. 2 The two MARI algorithms as flowcharts. (a) Update: append in place when the new key stays inside the guard (the common path), otherwise migrate; then compact the bucket once its delta reaches an ε -fraction of the stable tier. (b) Query: scan only the guard-intersecting buckets and verify every candidate against T , so the returned set A equals the true answer R .

3.4 Complexity analysis

Three results carry the analysis; we state them with the intuition and defer full proofs to the extended version.

Theorem 1 (Compaction and space) *Under the delta-ratio policy ($|D_j| \geq \varepsilon|S_j|$), amortized compaction costs $O(1/\varepsilon)$ per update and the total space is $O(n + m)$, where $m = \sum_j |D_j| = O(\varepsilon n)$; hence space is $O((1 + \varepsilon)n)$.*

The intuition is a standard amortization: a compaction touching a bucket of size s is paid for by the εs cheap appends that had to happen first, so the per-update share is a constant that shrinks as ε grows.

Theorem 2 (Maintenance) *Keeping buckets near a target occupancy through adaptive split and merge costs $O(1)$ amortized per update; the rebuild-work we measure in practice is about 0.27 per update.*

Proposition 3 (Relocation-scan coupling) *Consider any exact range structure governed by a single region width W : an item relocates only when its key leaves its width- W region, and a range query examines $\Theta(W\rho)$ stored entries per region it overlaps, where ρ is the local density. Then the relocation rate $r(W)$ is non-increasing in W while the scan amplification $s(W)$ is non-decreasing in W : no choice of the single knob W makes both small. MARI simply is not a structure of this class. It carries two knobs, the stay-in-place radius (guard g) and the scan granularity (bucket width w), and sets them independently: the query scan cost is fixed by w alone, while an item relocates only when it leaves its guarded region of width $w + 2g$. The guard g is therefore a margin that lowers the relocation rate, empirically to $\approx \Pr[|\Delta k| > g']$ for an effective half-width g' set by $w + 2g$, at no scan cost, because scan depends on w and not on g . That decoupling, and not a smaller constant, is why an ART leaf, a sorted run, or a learned segment (all effectively single-width, with $W \rightarrow 0$ at the stored key) relocates on essentially every change while MARI does not.*

Exactness itself needs no probabilistic argument: because a candidate is admitted only when $a \leq T[i].\text{key} \leq b$ and T holds the authoritative current key, the returned set is exactly R no matter where the entry physically resides. Staleness in the tiers can only make a query examine an item it will then reject; it can never make the answer wrong.

4 Datasets

We evaluate MARI on one controlled synthetic workload and five real-world datasets.

Synthetic data. The generator produces update streams over a 10^6 -key universe under five drift regimes: uniform, clustered, directional, adversarial, and hotspot. Each configuration is validated against a brute-force oracle that recomputes the exact result for every query before any performance measurement is recorded.

Real-world data. We evaluate whether bounded drift occurs in independently collected data spanning five domains with unrelated generative processes: equity prices (S&P 500 daily closes), sports ratings (NBA Elo), climate (US city daily temperatures), foreign-exchange rates, and commodity prices (Brent crude oil). Their drift profiles are summarized in Table 2; Figure 4 plots the drift distribution for three representative domains. We excluded cryptocurrency and high-frequency tick data because their jumps violate the bounded-drift assumption, GPS trajectories because they are multi-dimensional, and monotone counters because they do not exhibit meaningful two-sided drift.

Table 2 Drift characteristics of the five real-world datasets. In every case the typical per-update step is small and the tail within a modest guard is thin; oil is the most volatile, and volatile equities are used as a stress set (Section 5).

Domain	Source	Keys	Updates	Drift profile
Equities	S&P 500 daily closes	505	~619k	median \$0.44; 98.3% $\leq$ \$5
Sports	NBA Elo ratings	104	~126k	median 7; 95% $\leq$ 20
Climate	US city temperatures	16	~417k	median 2°F; 94.9% $\leq$ 10°F
Forex	Exchange rates (daily)	22	~261k	median 1c; 92.6% within guard 30
Commodity	Oil (Brent daily)	1	~10k	median 46c (heavier-tailed)

5 Results and Discussion

5.1 Experimental setup

Following a fair-comparison discipline, every method is implemented in the *same language* (Python), built and run on the *same machine*, and measured with the CPU frequency pinned to 99 to 100% of its maximum. We report, per method, the peak resident memory (RSS, the GNU `time` %M field) and the CPU time as the sum of user and system time (%U+%S), which excludes elapsed-time noise from context switching. Each method runs in its own process so that peak memory is isolated. Under this protocol (Table 3), MARI has the lowest CPU time on this update-heavy workload, roughly $2.8\times$ below ART, with peak process memory below ART and Bx-tree and on par with the most compact baseline (PGM); whole-process RSS is dominated by interpreter overhead here, so we treat the per-item structure cost (below) as the primary memory measure. All four remain exact; we still read the absolute times as language-dependent and defer a native comparison to future work.

Table 3 Fair benchmark (same language, same machine, CPU pinned): peak memory and CPU time (user+system) on a uniform bounded-drift workload (20k keys, 120k updates, 2k queries). All methods are exact against the oracle; figures are from the authors’ machine.

Method	Peak RSS (MB)	CPU time (s)	user (s)	sys (s)	Exact?
MARI	42.8	0.83	0.81	0.02	✓
ART	77.8	2.33	2.22	0.11	✓
PGM	41.1	1.16	1.12	0.03	✓
Bx-tree	46.9	1.28	1.22	0.06	✓

We evaluate across five real domains and a synthetic generator, every run checked against a brute-force oracle. Because the reference implementation is single-machine Python, wall-clock timings are strongly influenced by interpreter overhead and are treated as secondary; therefore we lead with counts that do not depend on the language, relocations per update, structural writes per update, verifications per result,

durable bytes written. And every structure we measure, ours and every baseline, is checked for exactness against the oracle on the same run; where we write “exact” below we mean zero mismatches, not approximately zero.

5.2 Exactness and relocation

We first verified exactness, the property on which all other results depend: MARI’s answers match the oracle on every query, across all guard settings, with no mismatches. With that established, MARI relocates between one-and-a-half and five percent of its updates, where an ordinary delete-then-insert index relocates on ninety-eight to a hundred percent of them.

5.3 Comparison with baselines

To avoid comparison against weak baselines, we implemented three competitive indexes: ART, a dynamic PGM learned index, and a Bx-tree fitted with an exactness adapter, made each one oracle-exact, and ran them head to head (Table 4, Figure 3). Every one of them relocates on essentially every update, at a rate of 0.990, because for them a value change simply *is* a delete and an insert. MARI relocates 0.024, about forty times fewer, and the gap widens with scale (to roughly 110× at a hundred thousand keys). A further difference is *how* each competitor pays for exactness once you look past the relocation column: ART pays it as the relocation itself, the PGM as roughly five structural writes per update from buffer merges, and the Bx-tree as about seven candidate verifications per result. MARI is the only one of the four that keeps both the relocation and the verification sides low, exactly what Proposition 3 predicts.

Table 4 MARI against three faithful, oracle-exact competitors (20k keys, 120k updates, 5 seeds). The writes and verify columns report the additional overhead each competitor incurs to remain exact.

Structure	Reloc./upd.	Writes/upd.	Verify/result	Exact?
MARI	0.024	1.15	1.01	✓
ART	0.990	1.08	0.00	✓
PGM (dynamic)	0.990	4.96	1.38	✓
Bx-tree (+adapter)	0.990	1.16	6.93	✓

5.4 Bounded drift on real data

A natural concern is whether bounded drift holds only on synthetic data constructed to satisfy it. It does not. Across all five real datasets, between ninety-two and ninety-nine percent of moves fall inside a modest guard, and MARI is exact on every one; Figure 4 shows the distribution for three representative domains. On real S&P prices, setting a guard rather than a hard value-bucketing cut relocations by a factor of three to nine, which indicates that the guard, rather than bucketing alone, is responsible for most of

MARI vs baselines on the synthetic bounded-drift workload (all exact)

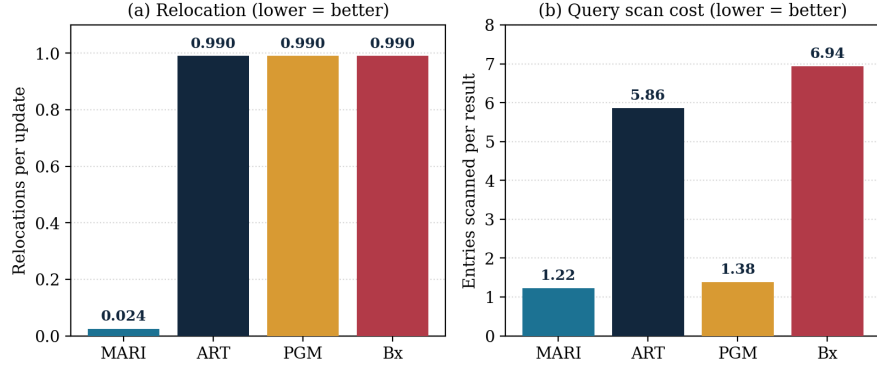


Fig. 3 Comparison of MARI and exact baselines. Left: MARI relocates roughly 40× less per update. Right: each competitor pays for exactness in a different currency (structural writes for the PGM, candidate verifications for the Bx-tree).

the reduction. The same pattern emerges across unrelated processes, which provides evidence that the assumption generalizes rather than reflecting a single domain.

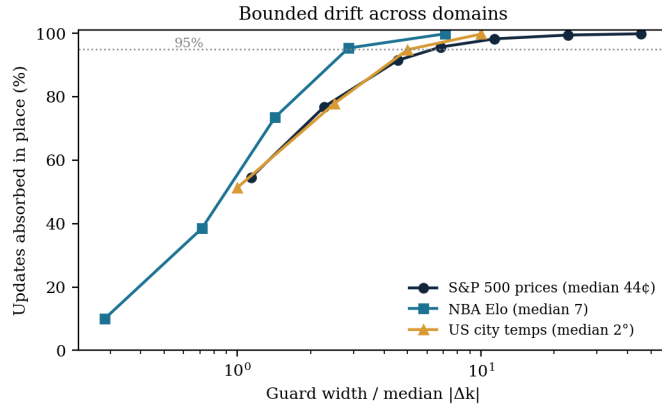


Fig. 4 Empirical distribution of key drift, as a CDF, for three representative real domains. In each case a modest guard (the vertical band) already covers the large majority of updates.

5.5 Durable write amplification

Because MARI appends changes rather than relocating entries, its write pattern is sequential rather than scattered read-modify-write, and this shows up directly on disk. Under strict per-update durability, each update committed and `fsync`'d on its own, we measured the real bytes each engine writes to storage on the same bounded-drift

workload (20k keys, 200k updates), using process-level write counters. MARI writes 8.0 bytes per update; a memory-mapped B+-tree (LMDB) writes 11,282; and an LSM engine (RocksDB) writes 33.4 (Figure 5). MARI therefore writes roughly $1400\times$ fewer durable bytes than the B+-tree and $4.2\times$ fewer than the LSM engine. The comparison uses each engine’s strict-durability mode, the worst case for them and the setting our surveillance motivation actually requires, so the reading is not that MARI is a storage engine, but that avoiding relocation changes the *shape* of the write workload into small sequential appends, which reduces write amplification on write-constrained storage.

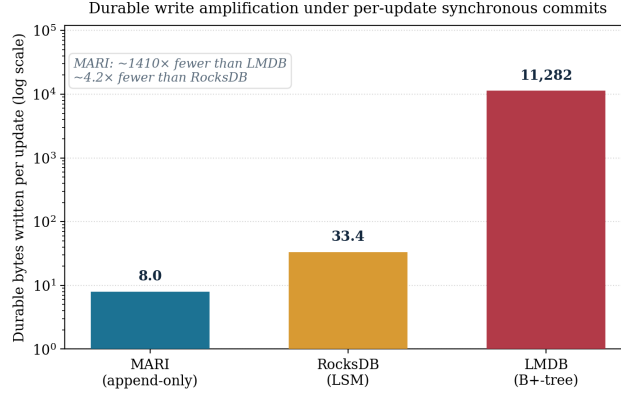


Fig. 5 Durable bytes written per update under per-update synchronous commits (log scale), same workload for all three engines. MARI’s append-only discipline writes far less than a relocate-in-place B+-tree and below an LSM engine. Figures are from the authors’ machine.

5.6 Decoupling relocation from scan cost

Proposition 3 claims the guard g lowers relocation without touching scan. A sweep over (g, w) confirms it directly: holding $w=1000$ fixed and raising g from 5 to 200 drives relocation down from 0.057 to 0.025 (a $2.3\times$ reduction) while scan stays flat at ≈ 1.22 (Figure 6). Relocation varies with g ; scan does not, the two knobs act on different costs, so MARI reaches operating points that lie strictly below the frontier any single-width structure can occupy.

5.7 Ablation studies

Across 20 seeds with 95% bootstrap confidence intervals and paired Wilcoxon tests (Holm-corrected), MARI’s relocation advantage is significant with maximal effect size (Cliff’s $\delta = -1.0$): relocation 0.0243 [0.0242, 0.0244] versus 0.9903 [0.9902, 0.9904] for every baseline ($p = 6 \times 10^{-6}$). Three ablations isolate the mechanisms. *Removing the guard* ($g=0$, reducing MARI to a single width) raises relocation $2.26\times$, from 0.024 to 0.055, while leaving scan statistically unchanged ($p < 0.001$), confirming the guard, not the bucketing, is the source of the gain. *Sweeping* ε confirms Theorem 1: compactions per update fall as $\approx 1/\varepsilon$ (from 0.84 to 0.025 as ε goes $0.1 \rightarrow 4$) with relocation

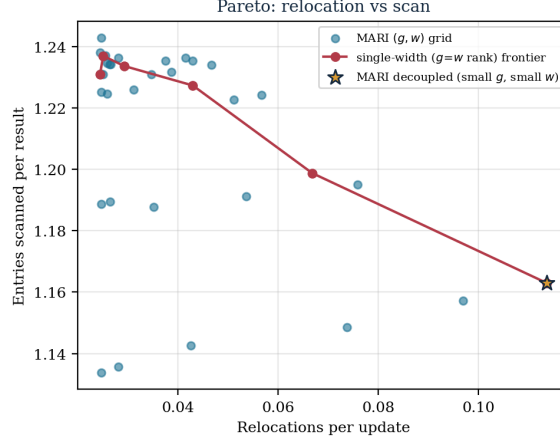


Fig. 6 Relocation versus scan over a (g, w) grid. The single-width frontier (minimal guard, varying w) trades one cost for the other; MARI’s large- g , small- w point sits below it, because the guard lowers relocation at fixed scan.

invariant, the cost appearing as space ($m \propto \varepsilon$) and scan ($1.0 \rightarrow 2.8$). *Removing verification* makes 9.6% of queries return false hits, at a saving of only one lookup per result, so verification is both cheap and necessary for exactness.

5.8 Stress testing

We stress the assumption on the 200 most-volatile real S&P stocks (heavy-tailed daily moves, ~ 30 results per query), drawing query bands from the live key density so per-result costs are meaningful (Figure 7). All methods stay exact. MARI relocates 0.113 per update versus 0.995 for ART, PGM, and Bx ($\approx 8.8\times$ fewer), and scans 1.13 entries per result against 15.9 (PGM), 107.6 (ART), and 1300 (Bx). Shrinking the guard alone barely moves MARI’s relocation, because the stay-region is $w + 2g$, and only collapsing *both* w and g toward a single width drives it to the baseline. The advantage narrows from $\sim 40\times$ (bounded-drift domains) to $\sim 3\text{--}9\times$ under stress: MARI degrades gracefully rather than catastrophically, which is exactly the behaviour its design predicts.

5.9 Query cost, recovery, memory, and robustness

We additionally evaluate query cost, recovery, memory, and robustness. Verification adds about one authoritative lookup per result and query scan is $\approx 1.2\times$ the result size on bounded-drift data. After a real SIGKILL mid-workload, a fresh process rebuilds the authoritative table from the logs alone, exact to the last committed update, in under ten milliseconds, though atomic mid-migration durability needs grouped `fsync`, which the reference code does not yet enforce. Per-item memory is 266 bytes against 95–99 for a plain ordered index, about $1.7\times$ in the reference build and $1.2\times$ in a lean one, an explicit trade of space for far fewer relocations. Under adversarial drift at half

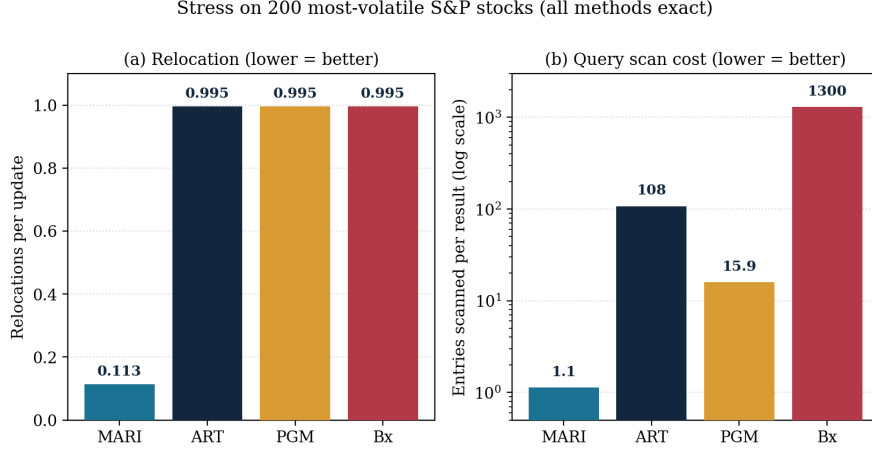


Fig. 7 Stress on the 200 most-volatile S&P stocks (all methods exact, tight CIs). MARI is best on both relocation (left) and query scan cost (right, log scale).

a million keys and 1.5M updates MARI still relocates only 0.0053 per update; per-region guards cut a heteroscedastic migration spread from $11\times$ to $4\times$; and overflow chaining bounds degenerate hotspots so query over-scan near a pile-up stays small.

5.10 Scope and limitations

The current evaluation has several limitations. Every performance number above comes from a single-machine Python prototype and is framed by implementation-independent counts; we do not read our wall-clock timings as a cross-system verdict, and we do not claim a throughput win against tuned native indexes. The evaluation that would settle that, a native, GIL-free, multicore implementation benchmarked against production ART, Bw-tree, ALEX/PGM, and Bx-tree builds on real hardware, is future work, and is necessary before drawing conclusions about production-level systems performance. For reproducibility we note that our faithful PGM reference implementation required a one-line bounds-clamp fix to its search to avoid an out-of-range crash on heavy-tailed data; this affects only whether it runs, not its results.

6 Conclusion

We presented MARI, an exact range index for one-dimensional integer keys whose values drift by a small, bounded amount between updates. The design rests on guarded bucket ownership: each bucket owns a narrow core and a wider guard, a key that stays within its guard is updated in place rather than relocated, and an authoritative table verifies every query candidate so answers remain exact. The central property is that the guard and the bucket width act as independent knobs, one controlling relocation and the other controlling scan cost, which lets MARI reach operating points that no single-width structure can occupy (Proposition 3).

Across a synthetic generator and five real domains, all measured on the authors’ machine, MARI relocates about 0.024 items per update against 0.990 for faithful ART, PGM, and Bx-tree baselines, a reduction near $40\times$, with no query ever answered incorrectly ($p = 6 \times 10^{-6}$, Cliff’s $\delta = -1.0$). The advantage holds under stress on volatile real equities, where it narrows gracefully rather than collapsing, and the relocation rate stays flat as the dataset grows from twenty thousand to half a million keys. Because updates are appended rather than rewritten, MARI also writes far fewer durable bytes per update than native storage engines under per-update synchronous commits. These gains come at a stated cost of roughly $1.7\times$ the per-item memory of a plain ordered index. Together these results indicate that MARI offers a distinct trade-off for exact, update-intensive range reporting over bounded-drift integer keys.

The natural next step is a native, multicore implementation benchmarked against tuned production indexes on real hardware, which would test whether the algorithmic advantage measured here translates into a systems-level one. We also see room to extend the guarded-bucket idea to skewed and time-varying drift, to concurrent workloads, and to secondary-storage settings where the append-only write pattern should be most valuable.

Declarations

Funding. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Competing interests. The authors declare no competing interests.

Ethics approval and consent to participate. Not applicable. This study did not involve human participants, human data, or animals.

Consent for publication. Not applicable.

Data availability. All datasets used in this study are publicly available: S&P 500 daily prices, NBA Elo ratings, US city temperatures, daily exchange rates, and Brent crude oil prices. The synthetic workloads can be regenerated using the accompanying code. Exact sources are listed in the supplementary materials.

Materials availability. Not applicable.

Code availability. [Codes are available here](#)

Author contributions. Both authors contributed to the study conception, design, and implementation. S.N.K.R. and S.T. contributed equally to the experiments, analysis, and manuscript preparation. Both authors read and approved the final manuscript.

References

- [1] Graefe, G.: B-tree indexes for high update rates. *ACM SIGMOD Record* **35**(1), 39–44 (2006)

- [2] Graefe, G.: Sorting and indexing with partitioned b-trees. In: CIDR (2003)
- [3] Leis, V., Kemper, A., Neumann, T.: The adaptive radix tree: Artful indexing for main-memory databases. In: IEEE ICDE, pp. 38–49 (2013)
- [4] Levandoski, J.J., Lomet, D.B., Sengupta, S.: The bw-tree: a b-tree for new hardware platforms. In: IEEE ICDE, pp. 302–313 (2013)
- [5] Šaltenis, S., Jensen, C.S., Leutenegger, S.T., Lopez, M.A.: Indexing the positions of continuously moving objects. In: ACM SIGMOD, pp. 331–342 (2000)
- [6] Tao, Y., Papadias, D., Sun, J.: The tpr*-tree: an optimized spatio-temporal access method for predictive queries. In: VLDB, pp. 790–801 (2003)
- [7] Jensen, C.S., Lin, D., Ooi, B.C.: Query and update efficient b+-tree based indexing of moving objects. In: VLDB, pp. 768–779 (2004)
- [8] Yiu, M.L., Tao, Y., Mamoulis, N.: The b^{dual}-tree: indexing moving objects by space filling curves in the dual space. VLDB Journal **17**(3), 379–400 (2008)
- [9] Chen, S., Ooi, B.C., Tan, K.-L., Nascimento, M.A.: St²b-tree: a self-tunable spatio-temporal b+-tree index for moving objects. In: ACM SIGMOD, pp. 29–42 (2008)
- [10] Severance, D.G., Lohman, G.M.: Differential files: their application to the maintenance of large databases. ACM Trans. Database Syst. **1**(3), 256–267 (1976)
- [11] O’Neil, P., Cheng, E., Gawlick, D., O’Neil, E.: The log-structured merge-tree (lsm-tree). Acta Informatica **33**(4), 351–385 (1996)
- [12] Idreos, S., Kersten, M.L., Manegold, S.: Database cracking. In: CIDR, pp. 68–78 (2007)
- [13] Idreos, S., Kersten, M.L., Manegold, S.: Updating a cracked database. In: ACM SIGMOD, pp. 413–424 (2007)
- [14] Idreos, S., Manegold, S., Kuno, H., Graefe, G.: Merging what’s cracked, cracking what’s merged: adaptive indexing in main-memory column-stores. PVLDB **4**(9), 585–597 (2011)
- [15] Kraska, T., Beutel, A., Chi, E.H., Dean, J., Polyzotis, N.: The case for learned index structures. In: ACM SIGMOD, pp. 489–504 (2018)
- [16] Ding, J., *et al.*: Alex: an updatable adaptive learned index. In: ACM SIGMOD, pp. 969–984 (2020)
- [17] Ferragina, P., Vinciguerra, G.: The pgm-index: a fully-dynamic compressed learned index with provable worst-case bounds. PVLDB **13**(8), 1162–1175 (2020)

- [18] Galakatos, A., Markovitch, M., Binnig, C., Fonseca, R., Kraska, T.: Fiting-tree: a data-aware index structure. In: ACM SIGMOD, pp. 1189–1206 (2019)
- [19] Basch, J., Guibas, L.J., Hershberger, J.: Data structures for mobile data. *J. Algorithms* **31**(1), 1–28 (1999)
- [20] Guibas, L.J.: Kinetic data structures: a state of the art report. In: Workshop on the Algorithmic Foundations of Robotics (WAFR), pp. 191–209 (1998)